# Integration guide: Retro Horse Race (platform edition)

For the platform / operator that wants to offer the game to its players. It explains the integration as built today, how a session and a bet flow, what the platform must implement, and, at the end, **what is missing or weak and should be improved**. The wire format of the wallet calls is in `WALLET_API.md`; deployment is in `DEPLOY.md`.

## 1. Integration model

| Item | Today |
|---|---|
| Launch | Redirect. The player's browser leaves the platform and opens `https://GAME_HOST/?token=<one-time token>`. No iframe / embed mode. |
| Wallet | Seamless (single) wallet. The platform keeps the money; the game debits each bet and credits each win through the platform's HTTP API. |
| Currency | One currency per game server (`CURRENCY`), amounts are integers in minor units (centavos). |
| Game rounds | One shared continuous table: bet (25 s) -> draw -> 6 s countdown -> race (30 s) -> results (10 s) -> next round. All players see the same race. |
| Result | Drawn by the game server (crypto RNG, committed with SHA-256 before betting closes, seed revealed after the race). Nobody, including the operator, can set the result in live mode. |
| Payout | Fixed odds board (shuffled each race). Payout = stake x odds, floor to a minor unit. Long-run RTP is an operator setting, 88% to 99%. |

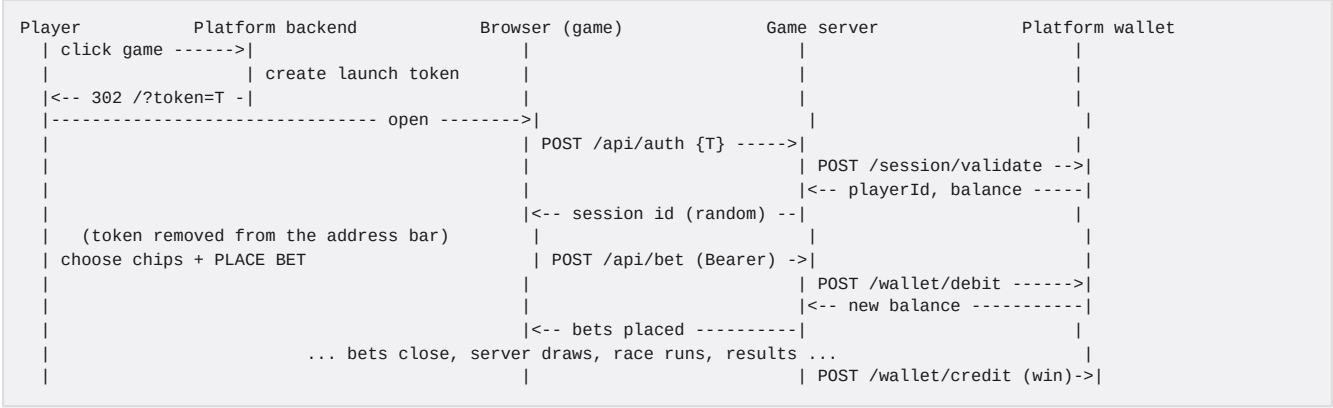
## 2. Roles

- **Platform backend**: creates launch tokens, implements the four wallet endpoints, keeps the player's balance.
- **Game server** (this repo): serves the game, runs rounds and draws, calls the wallet, keeps a ledger (`data/ledger.jsonl`).
- **Operator**: runs the game server and uses `/controller` (report and RTP setting).

## 3. Setup steps for the platform

1. **Deploy the game server** in live mode with the HTTP wallet (see `DEPLOY.md`): `GAME_MODE=live WALLET=http WALLET_URL=https://platform.example.com/horse-api WALLET_KEY=<secret> CONTROLLER_KEY=<operator key> CURRENCY=PHP RTP=90 node server.js` Put it behind HTTPS. Keep `/controller`, `/api/admin`, `/api/report`, `/api/command` reachable by the operator only.
2. **Implement the wallet API** (section 5).
3. **Add the launch button** in the lobby: your backend creates a random one-time token for the logged-in player and redirects the browser to `https://GAME_HOST/?token=<token>`.
4. **Test locally** without a platform: `GAME_MODE=live WALLET=mock node server.js`, open `http://localhost:8080/?token=demo-alice`.
5. **Reconcile** daily: compare `data/ledger.jsonl` (match on `txId`) with the platform's wallet transactions.

## 4. Flow of a session and a bet




Session details: the game server turns the launch token into its own random 192-bit session ID, valid for up to 12 hours (`sessionTtlMs`). It stores only a hash of the ID, in memory and in `data/sessions.json`, so a restart does not end sessions. The browser keeps the ID in `sessionStorage` and sends it as `Authorization: Bearer <session>`. Closing the tab ends the player's session on that device; the player launches again from the lobby. Balances and bets are not lost on a restart (money is on the platform; bets are recovered from the ledger and settled or refunded).

## 5. What the platform implements

All calls are `POST`, JSON, signed with HMAC (`X-Timestamp`, `X-Nonce`, `X-Signature`), amounts in minor units. Full detail in `WALLET_API.md`.

| Endpoint | Request | Reply |
|---|---|---|
| `/session/validate` | `{token}` | `{ok, playerId, currency, balance, returnUrl}` plus optional `brandId`, `decimals`, `limits`, `realityCheckMinutes`, `blocked`; the token is single use |
| `/wallet/balance` | `{playerId}` | `{ok, balance}` |
| `/wallet/debit` | `{playerId, amount, txId, meta: {raceNo, kind:"bet"}}` | `{ok, balance}` or HTTP 402 `INSUFFICIENT_FUNDS` |
| `/wallet/credit` | `{playerId, amount, txId, meta: {raceNo, kind:"win"\|"refund"}}` | `{ok, balance}` |
| `/wallet/rollback` | `{playerId, amount, txId, rollbackTxId, meta}` | `{ok, balance, status}`; cancels the debit `txId` and refuses it if it arrives later |

Rules the wallet must follow:

- **Idempotent on `txId`**: the same `txId` twice is applied once and returns the same reply.
- A debit that times out may or may not have been applied. The game retries once with the same `txId`; if it still fails the bet is rejected, logged as `debit_unknown`, and rolled back through `/wallet/rollback` (retried until confirmed).
- Failed credits (wins, refunds) are retried forever with back-off.
- Serve the wallet API over HTTPS only, keep `WALLET_KEY` secret.

## 6. Settings the operator controls

| Setting | Where | Notes |
|---|---|---|
| `RTP` (88 to 99%) | env at start, `/controller` while running | Takes effect from the next race. The odds never change; the three cheapest pairs absorb the difference. |
| `ODDS` / `ODDS_FILE` | env | Change the odds table (a restart is needed). |
| `BET_MS`, `MIN_BET`, `MAX_BET`, `MAX_ROUND_STAKE` | env | Minor units. Max bet is per pair, max round stake is per player per race. |
| `CURRENCY`, `WALLET_TIMEOUT_MS` | env | |

## 7. Security as built

- Browser never sends balances or payouts; stakes are validated server-side (integer, min/max, 15 valid pairs, per-pair and per-round limits).
- Launch token is one time and removed from the URL; `Referrer-Policy: no-referrer` on pages.
- Results come from the server's RNG with a SHA-256 commit shown before betting closes and the seed revealed after the race.
- Wallet requests are HMAC-signed with timestamp and nonce; the mock wallet cannot run in production; unknown debits are rolled back; sessions are stored hashed and survive restarts.
- Players can be limited (bet limits, daily loss limit), blocked and shown reality checks; every bet is in a per-player history with a verifiable draw proof.
- Bet requests are rate limited (12 per 5 s per session); the controller cannot change results in live mode.

## 8. Improvements to consider (gaps today)

Ordered by importance for a real-money launch.

### Must have before real money

1. **Signed wallet requests. DONE.** Every wallet call carries `X-Timestamp`, `X-Nonce` and an HMAC-SHA256 `X-Signature` (`server/sign.js`, spec in `WALLET_API.md`). The platform rejects bad signatures, stale timestamps and replayed nonces, and should also allowlist the game server's IP. `WALLET_AUTH=bearer` keeps the old mode for platforms that cannot verify yet.
2. **Block the mock wallet in production. DONE.** In live mode the play-money wallet needs `ALLOW_MOCK_WALLET=1` and is always refused when `NODE_ENV=production`; a `WALLET_KEY` shorter than 16 characters is refused too.
3. **Rollback of unknown debits. DONE.** When a debit times out twice the bet is rejected and the game calls `POST /wallet/rollback` for that `txId`, retrying with back-off (also after a restart) until the platform confirms. The platform must also refuse a late debit with a cancelled `txId`. If the platform has no rollback endpoint the case is left in the ledger for manual reconciliation. A status endpoint is still an option if you prefer it to rollback.
4. **Persistent sessions. DONE.** Sessions are kept in `data/sessions.json` (ids stored hashed, file mode 0600), so a restart or deploy does not log players out. Expired sessions are not restored. Operators can end a player's sessions with the controller command `closeSessions` (`POST /api/command {cmd:"closeSessions", playerId}`).

5. **Independent security review and RNG/math certification** by an accredited lab if the operator is licensed. (Not something the code can provide; still open.)

### Should have

1. **Round and bet IDs in every wallet call. DONE.** Debit, credit and rollback carry `gameId`, `transactionType`, `roundId`, `betId` / `betIds`, `refTxId`, `currency` and `brandId` at the top of the body (and in `meta`).
2. **Player history and draw proof. DONE.** `GET /api/history` (player session), `GET /api/admin/history?playerId=` (operator) and `GET /api/round/<n>` (public proof: commit, seed, odds, result). The betting panel has a MY HISTORY list. History is rebuilt from the ledger after a restart.
3. **Operator-driven limits. DONE.** `/session/validate` may return `limits` (min / max bet, max per race, daily loss limit) and `blocked`. A daily loss limit blocks bets that would take the player's net loss of the UTC day over it. Operator commands `closeSessions`, `blockPlayer` and `unblockPlayer` end sessions and keep a player out (self-exclusion).
4. **Reality-check hook. DONE.** `realityCheckMinutes` (per player or `REALITY_CHECK_MIN`) shows time played and the session result every N minutes, with CONTINUE and LEAVE (to `returnUrl`). The BACK TO LOBBY link is always there.
5. **Server-to-server events. DONE.** Signed `round.settled` and `big.win` webhooks (`WEBHOOK_URL`), at least once, stable `id`, retried with back-off and recovered after a restart.
6. **Multi-currency and multi-brand. DONE.** Currency, decimals and brand come per session; per-currency limit tables (`CURRENCY_LIMITS`); the report shows staked and paid per currency. Currencies are not converted: the platform keeps one wallet per currency.
7. **Bonus / free-bet support.** Today there is only real-money debit and credit.

### Nice to have

1. **Embed mode.** An iframe launch with a `postMessage` API (balance updated, round result, close game). The game already emits `countdown`, `start` and `result` events to a parent window, but the betting panel and session flow are built for redirect.
2. **Tamper-evident ledger. DONE.** Every ledger line is hash-chained (`node scripts/verify-ledger.js`); a broken chain is detected at start-up, raises an alert and turns `/healthz` red. Each `round.settled` webhook carries the newest hash (`ledgerHash`), so the platform can anchor the ledger outside the game server. Shipping the file to an append-only store is still up to the operator.
3. **Monitoring. DONE.** `/metrics` (Prometheus), `/healthz` that reflects the real state, a Health and alerts panel in the controller, and built-in alerts to a webhook: stuck wins / refunds / rollbacks, failing or slow wallet, stalled round loop, broken ledger, payouts drifting from what the odds predict, low disk.
4. **High availability. DONE (active / standby).** A lease file keeps exactly one instance running the races; a standby takes over in about 10 to 15 seconds from the same data directory, replaying the ledger; graceful shutdown on deploys. This is failover, not load sharing: still one table, one process.
5. **Aggregator adapter.** A small adapter in `server/wallet.js` if the platform sits behind a casino aggregator with its own API format.
6. **Localisation and currency formatting** per player, and a launch parameter for language.

## 9. Questions to settle with the platform

1. Redirect only, or do you also need an iframe embed?
2. Which wallet API do you already have? (We can adapt `server/wallet.js` to it.)
3. Do you need bonus, multi-currency or per-player limits at launch?

4. What must the player history show, and how do you want to reconcile (daily file, API)?
5. Who hosts the game server, and who operates `/controller`?
6. Is the operator licensed, and does that require lab certification of the RNG and the game math?